HTTP modules handle the cross-cutting concerns of applications, such as security, logging, and session management. They run in response to the life-cycle events that a request progresses through when it's received by the server. Examples of events include `BeginRequest`, `AcquireRequestState`, and `PostAcquireRequestState`.

This approach works, but sometimes it's tricky to reason about which modules will run at which points. Implementing a module requires relatively detailed understanding of the state of the request at each individual life-cycle event.

The middleware pipeline makes understanding your application far simpler. The pipeline is defined completely in code, specifying which components should run and in which order. Behind the scenes, the middleware pipeline in ASP.NET Core is simply a chain of method calls, with each middleware function calling the next in the pipeline.

That's pretty much all there is to the concept of middleware. In the next section, I'll discuss ways you can combine middleware components to create an application and how to use middleware to separate the concerns of your application.

4.2 Combining middleware in a pipeline

Generally speaking, each middleware component has a single primary concern; it handles only one aspect of a request. Logging middleware deals only with logging the request, authentication middleware is concerned only with identifying the current user, and static-file middleware is concerned only with returning static files.

Each of these concerns is highly focused, which makes the components themselves small and easy to reason about. This approach also gives your app added flexibility. Adding static-file middleware, for example, doesn't mean you're forced to have image-resizing behavior or authentication; each of these features is an additional piece of middleware.